

Weekly Topic - Compose Layout Basics

Tuesday, May 26, 2026 15:44

[Fundamentals of Compose Layouts and Modifiers - MAD Skills](#)



Compose...

Compose layout basics



Jetpack Compose makes it much easier to design and build your app's UI. Compose transforms state into UI elements, via:



Fundamentals of Compose
Android Developers



1. Composition of elements
2. Layout of elements
3. Drawing of elements



This document focuses on the layout of elements, explaining some of the building blocks Compose provides to help you lay out your UI elements.

Goals of layouts in Compose

The Jetpack Compose implementation of the layout system has two main goals:

- High performance (#performance)
- Ability to easily write custom layouts
(/develop/ui/compose/layouts/custom)

Note: With the Android View system, you could face some performance issues when nesting certain Views such as **RelativeLayout**. Since Compose avoids multiple measurements, you can nest as deeply as you want without affecting

when nesting certain Views such as **RelativeLayout**. Since Compose avoids multiple measurements, you can nest as deeply as you want without affecting performance.

Basics of composable functions

Composable functions are the basic building block of Compose. A composable function is a function emitting **Unit** that describes some part of your UI. The function takes some input and generates what's shown on the screen. For more information about composables, take a look at the [Compose mental model](#) ([/develop/ui/compose/mental-model](#)) documentation.

A composable function might emit several UI elements. However, if you don't provide guidance on how they should be arranged, Compose might arrange the elements in a way you don't like. For example, this code generates two text elements:

```
@Composable
fun ArtistCard() {
    Text("Alfred Sisley")
    Text("3 minutes ago")
}
a/com/example/compose/snippets/layouts/LayoutBasicsSnippets.kt#L47-L51)
```

Without guidance on how you want them arranged, Compose stacks the text elements on top of each other, making them unreadable:



Compose provides a collection of ready-to-use layouts to help you arrange your UI elements, and makes it easy to define your own, more-specialized layouts.

Standard layout components

Standard layout components

In many cases, you can just use [Compose's standard layout elements](#)

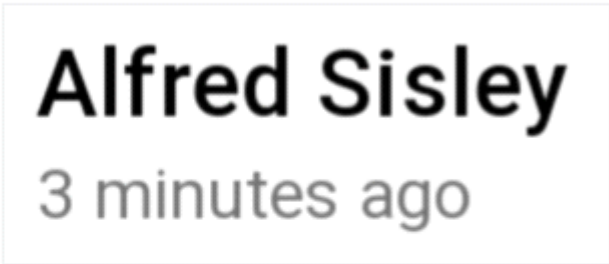
([/reference/kotlin/androidx/compose/foundation/layout/package-summary](#)).

Use `Column`

([/reference/kotlin/androidx/compose/foundation/layout/Column.composable#Column\(androidx.compose.ui.Modifier,androidx.compose.foundation.layout.Arrangement.Vertical,androidx.compose.ui.Alignment.Horizontal,kotlin.Function1\(androidx.compose.ui.Modifier,androidx.compose.foundation.layout.Arrangement.Vertical,androidx.compose.ui.Alignment.Horizontal,kotlin.Function1\)\)](#))
to place items vertically on the screen.

```
@Composable
fun ArtistCardColumn() {
    Column {
        Text("Alfred Sisley")
        Text("3 minutes ago")
    }
}
```

[a/com/example/compose/snippets/layouts/LayoutBasicsSnippets.kt#L55-L61](#)



Similarly, use `Row`

([/reference/kotlin/androidx/compose/foundation/layout/Row.composable#Row\(androidx.compose.ui.Modifier,androidx.compose.foundation.layout.Arrangement.Horizontal,androidx.compose.ui.Alignment.Vertical,kotlin.Function1\(androidx.compose.ui.Modifier,androidx.compose.foundation.layout.Arrangement.Horizontal,androidx.compose.ui.Alignment.Vertical,kotlin.Function1\)\)](#))
to place items horizontally on the screen. Both `Column` and `Row` support configuring the alignment of the elements they contain.

Similarly, use **Row**

([/reference/kotlin/androidx/compose/foundation/layout/Row.composable#Row\(androidx.compose.ui.Modifier,androidx.compose.foundation.layout.Arrangement.Horizontal,androidx.compose.ui.Alignment.Vertical,kotlin.Function1\)\(androidx.compose.ui.Modifier,androidx.compose.foundation.layout.Arrangement.Horizontal,androidx.compose.ui.Alignment.Vertical,kotlin.Function1\)\)](#))
to place items horizontally on the screen. Both **Column** and **Row** support configuring the alignment of the elements they contain.

```
@Composable
fun ArtistCardRow(artist: Artist) {
    Row(verticalAlignment = Alignment.CenterVertically) {
        Image(bitmap = artist.image, contentDescription = "
        Column {
            Text(artist.name)
            Text(artist.lastSeenOnline)
        }
    }
}
```

[a/com/example/compose/snippets/layouts/LayoutBasicsSnippets.kt#L65-L82](#)



Alfred Sisley
3 minutes ago

Use **Box**

([/reference/kotlin/androidx/compose/foundation/layout/Box.composable#Box\(androidx.compose.ui.Modifier,androidx.compose.ui.Alignment,kotlin.Boolean,kotlin.Function1\)\)](#))

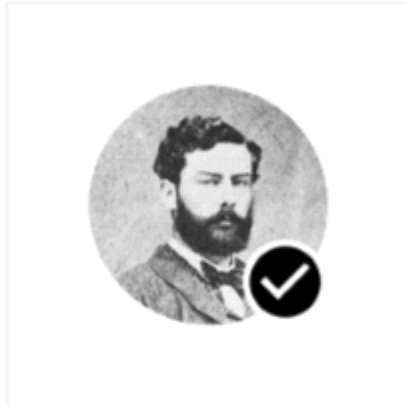
to put elements on top of another. **Box** also supports configuring specific alignment of the elements it contains.

```
@Composable
fun ArtistAvatar(artist: Artist) {
    Box {
        Image(bitmap = artist.image, contentDescription = "
```

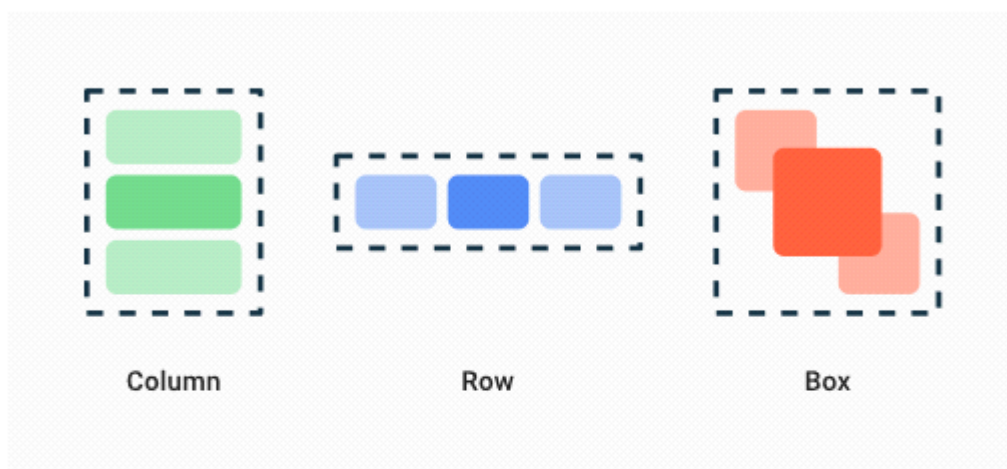
```

fun ArtistLayout(artist: Artist) {
    Box {
        Image(bitmap = artist.image, contentDescription = "
        Icon(Icons.Filled.Check, contentDescription = "Chec
    }
}
a/com/example/compose/snippets/layouts/LayoutBasicsSnippets.kt#L86-L92)

```



Often these building blocks are all you need. You can write your own composable function to combine these layouts into a more elaborate layout that suits your app.



Note: Compose handles nested layouts efficiently, making them a great way to design a complicated UI. This is an improvement from Android Views, where you need to avoid nested layouts for performance reasons.

To set children's position within a `Row`, set the `horizontalArrangement` and `verticalAlignment` arguments. For a `Column`, set the `verticalArrangement` and `horizontalAlignment`

`horizontalArrangement` and `verticalAlignment` arguments. For a `Column`, set the `verticalArrangement` and `horizontalAlignment` arguments:

```
@Composable
fun ArtistCardArrangement(artist: Artist) {
    Row(
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.End
    ) {
        Image(bitmap = artist.image, contentDescription = "
        Column { /*...*/ }
    }
}
```

com/example/compose/snippets/layouts/LayoutBasicsSnippets.kt#L96-L105



Alfred Sisley
3 minutes ago

The layout model

In the layout model, the UI tree is laid out in a single pass. Each node is first asked to measure itself, then measure any children recursively, passing size constraints down the tree to children. Then, leaf nodes are sized and placed, with the resolved sizes and placement instructions passed back up the tree.

Briefly, parents measure before their children, but are sized and placed after their children.

Consider the following `SearchResult` function.

```
@Composable
```


Briefly, parents measure before their children, but are sized and placed after their children.

Consider the following `SearchResult` function.

```
@Composable
fun SearchResult() {
    Row {
        Image(
            // ...
        )
        Column {
            Text(
                // ...
            )
            Text(
                // ...
            )
        }
    }
}
```

[om/example/compose/snippets/layouts/LayoutBasicsSnippets.kt#L109-L130](https://github.com/androidx/androidx/blob/master/compose/snippets/layouts/LayoutBasicsSnippets.kt#L109-L130)

This function yields the following UI tree.

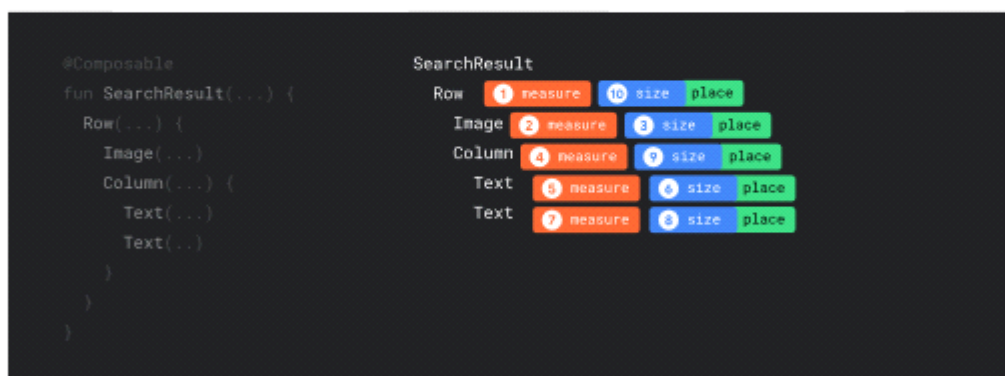
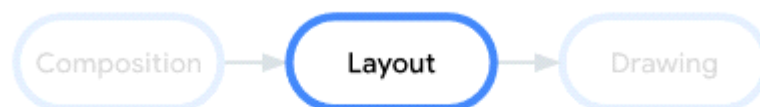
```
SearchResult
  Row
```

```
    Image
    Column
      Text
      Text
```

In the `SearchResult` example, the UI tree layout follows this order:

1. The root node `Row` is asked to measure.
2. The root node `Row` asks its first child, `Image`, to measure.

1. The root node **Row** is asked to measure.
2. The root node **Row** asks its first child, **Image**, to measure.
3. **Image** is a leaf node (that is, it has no children), so it reports a size and returns placement instructions.
4. The root node **Row** asks its second child, **Column**, to measure.
5. The **Column** node asks its first **Text** child to measure.
6. The first **Text** node is a leaf node, so it reports a size and returns placement instructions.
7. The **Column** node asks its second **Text** child to measure.
8. The second **Text** node is a leaf node, so it reports a size and returns placement instructions.
9. Now that the **Column** node has measured, sized, and, placed its children, it can determine its own size and placement.
10. Now that the root node **Row** has measured, sized, and placed its children, it can determine its own size and placement.



Performance

Compose achieves high performance by measuring children only once. Single-pass measurement is good for performance, allowing Compose to efficiently handle deep UI trees. If an element measured

once. Single-pass measurement is good for performance, allowing Compose to efficiently handle deep UI trees. If an element measured its child twice and that child measured each of its children twice and so on, a single attempt to lay out a whole UI would have to do a lot of work, making it hard to keep your app performant.

If your layout needs multiple measurements for some reason, Compose offers a special system, *intrinsic measurements*. You can read more about this feature in [Intrinsic measurements in Compose layouts](/develop/ui/compose/layouts/intrinsic-measurements) (/develop/ui/compose/layouts/intrinsic-measurements).

Since measurement and placement are distinct sub-phases of the layout pass, any changes that only affects placement of items, not measurement, can be executed separately.

Using modifiers in your layouts

As discussed in [Compose modifiers](/develop/ui/compose/modifiers) (/develop/ui/compose/modifiers), you can use modifiers to decorate or augment your composables. Modifiers are essential for customizing your layout. For example, here we chain several modifiers to customize the `ArtistCard`:

```
@Composable
fun ArtistCardModifiers(
    artist: Artist,
    onClick: () -> Unit
) {
    val padding = 16.dp
    Column(
        Modifier
            .clickable(onClick = onClick)
            .padding(padding)
            .fillMaxWidth()
    ) {
        Row(verticalAlignment = Alignment.CenterVertically)
        Spacer(Modifier.size(padding))
        Card(
            elevation = CardDefaults.cardElevation(defaultE
        ) { /*...*/ }
```

```
}  
}
```

com/example/compose/snippets/layouts/LayoutBasicsSnippets.kt#L134-L152)



Alfred Sisley

3 minutes ago



In the code above, notice different modifier functions used together.

- `clickable` makes a composable react to user input and shows a ripple.
- `padding` puts space around an element.
- `fillMaxWidth` makes the composable fill the maximum width given to it from its parent.
- `size()` specifies an element's preferred width and height.

Note: Among other things, modifiers play a role similar to that of layout parameters in view-based layouts. However, since modifiers are sometimes scope-specific, they offer type safety and also help you to discover and understand what is available and applicable to a certain layout. With XML layouts, it is sometimes hard to find out if a particular layout attribute is applicable to a given view.

layouts, it is sometimes hard to find out if a particular layout attribute is applicable to a given view.

Scrollable layouts

Learn more about scrollable layouts in the [Compose gestures documentation](#)

(/develop/ui/compose/touch-input/pointer-input/understand-gestures).

For lists and lazy lists, check out the [Compose lists documentation](#) (/develop/ui/compose/lists).

Responsive layouts

A layout should be designed with consideration of different screen orientations and form factor sizes. Compose offers out of the box a few mechanisms to facilitate adapting your composable layouts to various screen configurations.

Constraints

In order to know the constraints coming from the parent and design the layout accordingly, you can use a `BoxWithConstraints`. The [measurement constraints](#)

(/reference/kotlin/androidx/compose/foundation/layout/BoxWithConstraintsScope)

can be found in the scope of the content lambda. You can use these measurement constraints to compose different layouts for different screen configurations:

```
@Composable
fun WithConstraintsComposable() {
    BoxWithConstraints {
        Text("My minHeight is $minHeight while my maxWidth
    }
}
```

com/example/compose/snippets/layouts/LayoutBasicsSnippets.kt#L156-L161)

```
}  
com/example/compose/snippets/layouts/LayoutBasicsSnippets.kt#L156-L161)
```

Slot-based layouts

Compose provides a large variety of composables based on [Material Design](https://material.io/design/) (<https://material.io/design/>) with the `androidx.compose.material:material` dependency (included when creating a Compose project in Android Studio) to make UI building easy. Elements like [Drawer](https://material.io/components/navigation-drawer/) (<https://material.io/components/navigation-drawer/>), [FloatingActionButton](https://material.io/components/buttons-floating-action-button/) (<https://material.io/components/buttons-floating-action-button/>), and [TopAppBar](https://material.io/components/app-bars-top) (<https://material.io/components/app-bars-top>) are all provided.

Material components make heavy use of *slot APIs*, a pattern Compose introduces to bring in a layer of customization on top of composables. This approach makes components more flexible, as they accept a child element which can configure itself rather than having to expose every configuration parameter of the child. Slots leave an empty space in the UI for the developer to fill as they wish. For example, these are the slots that you can customize in a [TopAppBar](https://material.io/components/app-bars-top) (<https://material.io/components/app-bars-top>):



Composables usually take a `content` composable lambda (`content: @Composable () -> Unit`). Slot APIs expose multiple `content` parameters for specific uses. For example, [TopAppBar](https://material.io/components/app-bars-top)

allows you to provide the content for `title`, `navigationIcon`, and `actions`.

allows you to provide the content for `title`, `navigationIcon`, and `actions`.

For example, `Scaffold`

([https://developer.android.com/reference/kotlin/androidx/compose/material3/ScaffoldComposable#Scaffold\(androidx.compose.ui.Modifier,kotlin.Function0,kotlin.Function0,kotlin.Function0,kotlin.Function0,androidx.compose.material3.FabPosition,androidx.compose.ui.graphics.Color,androidx.compose.ui.graphics.Color,androidx.compose.foundation.layout.WindowInsets,kotlin.Function1\)](https://developer.android.com/reference/kotlin/androidx/compose/material3/ScaffoldComposable#Scaffold(androidx.compose.ui.Modifier,kotlin.Function0,kotlin.Function0,kotlin.Function0,kotlin.Function0,androidx.compose.material3.FabPosition,androidx.compose.ui.graphics.Color,androidx.compose.ui.graphics.Color,androidx.compose.foundation.layout.WindowInsets,kotlin.Function1)))

allows you to implement a UI with the basic Material Design layout structure. `Scaffold` provides slots for the most common top-level Material components, such as `TopAppBar`

(<https://material.io/components/app-bars-top/>), `BottomAppBar`

(<https://material.io/components/app-bars-bottom/>),

`FloatingActionButton`

(<https://material.io/components/buttons-floating-action-button/>), and

`Drawer` (<https://material.io/components/navigation-drawer/>). By using `Scaffold`, it's easy to make sure these components are properly positioned and work together correctly.



```
@Composable
fun HomeScreen(/*...*/) {
    ModalNavigationDrawer(drawerContent = { /* ... */ }) {
        Scaffold(
            topBar = { /*...*/ }
        ) { contentPadding ->
            // ...
        }
    }
}
```

[com/example/compose/snippets/layouts/LayoutBasicsSnippets.kt#L165-L176](https://developer.android.com/example/compose/snippets/layouts/LayoutBasicsSnippets.kt#L165-L176)

[Previous](#)

← [Overview](#) (/develop/ui/compose/layouts)

[Next](#)

[Modifiers](#) (/develop/ui/compose/modifiers) →

Content and code samples on this page are subject to the licenses described in the [Content License](#) (/license). Java and OpenJDK are trademarks or registered trademarks of Oracle and/or its affiliates.

Last updated 2026-05-19 UTC.

